

TRAFFIC SIGN RECOGNITION AND CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORKS

Abstract:

Traffic signs are really important for keeping everyone safe on the road. help drivers know how fast they can go, out for dangers, and other important stuff. This paper talks about a system that can recognize and figure out what different traffic signs mean in real-time. It has three parts: breaking down images, finding traffic signs, and sorting them out. The system uses special techniques to pick out red sections from pictures. It also uses Convolutional Neural Networks (CNN) to do the detecting, classify, and recognize parts to spot the message on traffic signs accurately.

Introduction:

Advanced Driver Assistance Systems (ADAS) are there to tell drivers what's up with traffic signals and warning signs they should pay attention to on the road. Sometimes drivers are lazy, not pay attention, or don't follow the rules. Our system wants to make driving safer by making sure drivers can see road signs clearly from far away. This way, drivers can avoid accidents and risky situations. For example, the system can warn drivers about places they shouldn't enter or help them slow down to keep things safe on the road.

Problems with TSR:

Traffic Sign Recognition (TSR) algorithms encounter three major issues. The image quality may suffer due to low resolution, harsh weather, and inadequate lighting. Additionally, signs might rotate incorrectly, get obscured, or fade away in certain scenarios. Moreover, applications such as ADAS face restrictions with limited memory and processing capabilities.



Fig 1 Difficulties facing in Traffic Sign Recognition (TSR) task: over-illumination, under illumination, rotation, occlusion and deterioration of the signs.

Problem Statement:

To enhance the performance of traffic sign recognition systems, Convolutional Neural Networks (CNN) are employed. CNN serves as a robust neural network method for classifying and identifying traffic signs effectively. This study focuses on designing a machine learning model to recognize German traffic signs using the dataset from the German Traffic Sign Benchmark (GTSB), coding the system in Python language.

Overview of Proposed Architecture:

This Convolutional Neural Network (CNN) architecture excels in several ways compared to other models

Feature Extraction: CNNs are recognized for their exceptional ability to extract spatial hierarchies of features from images. This model utilizes multiple convolutional layers, each increasing in complexity, enabling it to capture a wide range of features, from simple edges to intricate textures and shapes.

Standardization: By resizing all input images to 200x200 pixels, the model ensures uniformity in input dimensions, streamlining the training process and improving consistency across the dataset.

Advanced Techniques: Adding ReLU activations, batch normalization, and max pooling boosts the model's performance. ReLU adds non-linearity, batch normalization steadies and speeds up training, and max pooling trims computational complexity while preserving crucial features.

Optimization: The utilization of the Adam optimizer, renowned for its adaptive learning rate capabilities, aids in achieving quicker convergence and enhanced performance.

It improves accuracy by using convolutional layers, ReLU activations, batch normalization, and max pooling. This makes the model strong and accurate in tasks like classifying images. It's efficient too. Max pooling shrinks feature maps, reducing the need for lots of computing power. This makes the model work better without losing important details. And it's stable and fast. Batch normalization speeds up training by keeping learning in check. It also helps the model do well on new data. Lastly, it's versatile. The design lets it handle all kinds of images, not just the 43 classes it currently knows. This makes it useful for many different image problems. The model is structured for excellent performance in image classification, providing high accuracy, precision, recall, and F1 scores. Training is made easier with standardized input dimensions and advanced optimization techniques like Adam. Generalization is improved by using batch normalization and dropout (if included), preventing overfitting and ensuring good performance on new data. The model's ability to classify images accurately makes it applicable in real-world scenarios, such as medical image analysis and autonomous driving. The layer structure of this CNN sets it apart. Unlike traditional neural networks or simple machine learning algorithms, this CNN uses a multi-layered approach. With convolutional, pooling, and fully connected layers, each layer has a specific purpose in feature extraction and classification. Non-linearity and normalization play a crucial role in distinguishing this CNN. The incorporation of ReLU activations and batch normalization sets it apart from basic neural networks. These techniques enhance learning capabilities and stability. Adaptive optimization is another key feature. The Adam optimizer adjusts learning rates for each parameter. This improvement over static learning rate methods used in older models leads to better performance. Dimensionality reduction is also a unique aspect of CNNs. Max pooling reduces the size of feature maps and computational load, unlike traditional fully connected neural networks.

Introduction to Convolutional Neural Networks (CNN):

CNN is a deep-learning neural network model with distinct characteristics. It utilizes locally connected neurons and shared weight calculations while providing spatial or temporal sub-sampling. CNN typically consists of alternating Convolutional layers and max pooling layers. Each layer's input is derived from its preceding layer, forming a hierarchical feature extractor that transforms input images into feature vectors. The second part involves classifying the extracted feature vectors through fully connected layers, resembling a traditional feed-forward neural network structure.

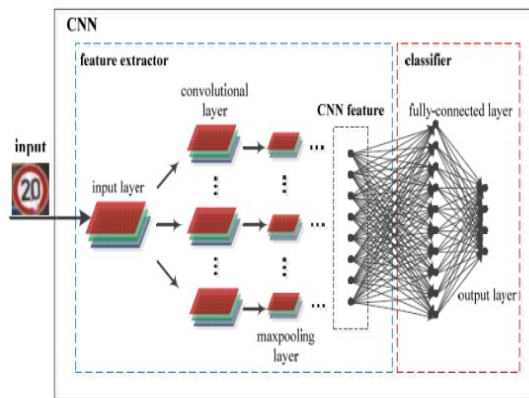


Fig. 2 CNN feature extractor architecture.

Related Work:

Existing Approaches:

A lot of traffic recognition algorithms split the problem into three stages: First, they do a rough segmentation to find where the signs are. Then, they figure out what category each sign falls under. Finally, they classify the exact content of the signs by using different machine-learning techniques. The first phase of segmentation isn't always necessary but is usually used with color images. Here's a quick look at some techniques used in the three Traffic Sign Recognition (TSR) stages of ELM. Some methods like [2,3] cut down on memory and processing needs by implementing tracking. They track candidates found across multiple frames to back up decisions or narrow down the search space and speed up performance. Some systems only check every n th frame to make the whole process faster.

Determining the Location using segmentation:

Traffic signs are painted in bright colors like red, blue, and yellow so that they're easily noticeable. This is why detection often starts with pre-segmenting the image to focus on specific areas and pick out Regions of Interest (ROIs). Instead of directly using RGB channel thresholds which can be affected by changes in lighting, the relationship between different RGB colors is usually considered. Color enhancement highlights red, blue, and yellow sections so that they can be identified separately. Filters for chromatics and achromatics help extract red borders and white areas inside speed limit and warning signs respectively. The HSI model (Hue Saturation Intensity) is preferred because it's not impacted by lighting changes even though it's computationally intensive due to its complex formulas.

Category Determination using Shape Detection

Lots of detection methods rely on edge detection which can be influenced by lighting variations. These algorithms recognize the shapes of traffic signs by taking advantage of their consistent patterns and symmetry. Circular and triangular shapes are identified using distance transforms (DT) and template matching (TM). Color Distance Transform differs by computing DT for each color channel independently. Matching DTs over edge images gives a smoother comparison that's more resistant to slight rotations while being sensitive to affine rotations and blockages. Support Vector Machines (SVMs) are trained on Distance-to-border (DtB) vectors

to categorize candidate ROI shapes. In [2], FFT signatures of signs are utilized for identifying relevant shapes that handle rotation and scaling well but struggle with occlusions.

The Hough Transform is commonly used to spot standard shapes like circles and triangles [4]. For quicker results, a simpler Radial Symmetry Detector method is used although it focuses mostly on circular signs. To fine-tune candidate selection, median filtering, dilation, and Confidence-Weighted Mean Shift are combined with Hough Transform outcomes for better accuracy. During detection, gradient orientation information is used along with comparing local edge orientations from fixed points with template orientations for classifying candidate signs properly. Edge Orientation Histograms are created within specific sub-regions which help in defining Regions of Interest (ROI) using Histograms of Oriented Gradients (HOG) feature embodying overall color properties concatenated with HOG descriptors calculated individually on each color channel offering features like scale-invariance, local contrast normalization, coarse spatial sampling as well as fine weight orientation binning benefits."

Methodology:

Data Processing and Transformations:

Resizing: Images are resized to a standard size of (200, 200) pixels. This standardization ensures consistent input dimensions across the dataset, which is essential for the model to learn effectively from the data.

Random Horizontal Flipping: This augmentation technique is applied to increase dataset variability. By randomly flipping images horizontally, the model is exposed to different orientations of the same object, enhancing its ability to generalize to unseen data and reducing overfitting.

ToTensor Transformation: Images are converted into PyTorch tensors, a data format compatible with PyTorch's computational graph framework. This conversion facilitates efficient computation, particularly when training on GPU, as tensors can be processed in parallel, speeding up training.

Normalization: Pixel values are normalized to have a mean of 0.5 and a standard deviation of 0.5. This normalization process ensures that pixel values are within a similar range, preventing certain features from dominating others during training. By standardizing the data scale and distribution, normalization aids in model convergence and stability during the optimization process.

Flowchart of the Proposed Architecture

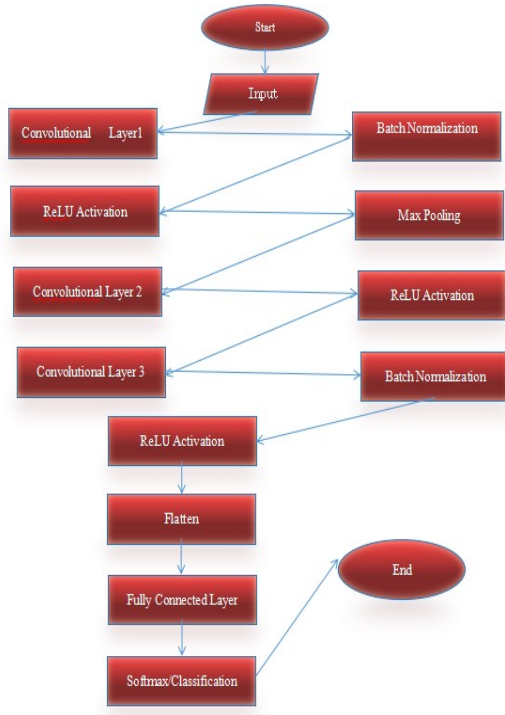


Fig 3: CNN Architecture Flowchart: This flowchart outlines a CNN architecture for image classification. It processes an image through convolutional layers (feature extraction) with increasing filter complexity, incorporating BatchNorm and ReLU for stability and non-linearity. Max pooling reduces dimensionality. The flattened output feeds into a fully connected layer for classification with Softmax activation.

Parameter setting

Used formula:

$$((W-k+2p)/s)+1=W$$

Here, W is the height or width size of the input image. in our case that is 200.

Now, as the result has to be equal to the size of the input image, we set the parameters for “k” kernel size to 3, padding size to 1 and stride size to 1.

Batch Normalization

$$\text{Mean} = \frac{(f_1 + f_2 + f_3 + \dots + f_N)}{N}$$

$$\text{Batch normalization} = \frac{(f_i - \text{Mean})}{\sqrt{\frac{\sum_{i=0}^N (f_i - \text{Mean})^2}{N}}}$$

For each feature in the input of a layer, batch normalization calculates the mean and variance across the mini-batch. It then normalizes the features by subtracting the mean and dividing by the standard deviation. Here, f_i is the i th feature. N is the number of features.

Convolutional Neural Network (CNN) Architecture

Input Layer: The input layer of the CNN represents the raw pixel values of the input images. Each image is typically represented as a multi-dimensional array of pixel values, where the dimensions correspond to height, width, and channels (e.g., RGB channels). In this case, the input images are resized to a standard size of (200, 200) pixels before being processed by the network. This standardization ensures uniformity in input dimensions across the dataset.

Convolutional Layers: The convolutional layers (conv1, conv2, and conv3) in the CNN architecture are fundamental for feature extraction from input images. These layers employ convolution operations with learnable filters or kernels to detect patterns and features across different spatial hierarchies. With convolutions, they capture both local and global information crucial for understanding the input data, facilitating the learning of meaningful representations of the images. The first convolutional layer (conv1) has 3 input channels (RGB channels) and 12 output channels, with a kernel size of 3x3, a stride of 1, and padding of 1. ReLU activation (relu1) and batch normalization (bn1) are applied, followed by max pooling (pool). Similarly, conv2 takes the output from conv1 with 12 input channels, producing 20 output channels. conv3 follows with 20 input channels (output from conv2) and 32 output channels. All convolutional layers use a ReLU activation function and include batch normalization (bn3) after conv3.

Batch Normalization: Batch normalization layers (bn1 and bn3) are employed after the convolutional layers to normalize the activations. This normalization process adjusts the mean and standard deviation of the activations, stabilizing and accelerating training.

ReLU Activation: ReLU activation functions (relu1, relu2, and relu3) introduce non-linearity into the network by zeroing out negative values. This non-linear activation function amplifies the model's capability to learn intricate patterns and gradients, crucial for effective feature learning and classification.

Max Pooling Layers: Max pooling layers (pool) are incorporated after the first convolutional layer to downsample feature maps. These layers reduce the spatial dimensions of the feature maps while retaining the most salient features, aiding in reducing computational complexity and enhancing the model's robustness to spatial translations in the input data.

Fully Connected Layer (Output Layer): At the end of the network, a fully connected layer (fc) serves as the output layer responsible for performing classification based on the extracted features. This layer aggregates and transforms the acquired features into a format conducive to making predictions across the 43 output classes. Through a series of linear transformations and non-linear activations, the fully connected layer maps the features to the respective output classes, facilitating accurate classification of the input images.

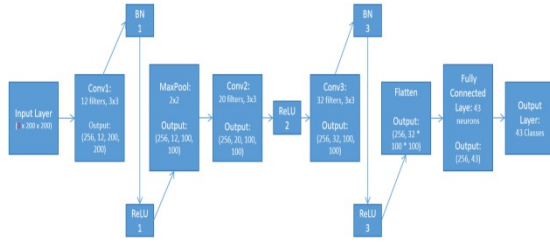


Fig 4: Step-by-Step Feature Extraction and Classification with CNN. This flowchart illustrates a Convolutional Neural Network (CNN) architecture for image classification. The input image traverses a series of convolutional layers (Conv1, Conv2, Conv3), each using filters to identify specific features (edges, shapes, textures). Batch normalization (BN) layers enhance stability, while ReLU activations introduce non-linearity. Max pooling (MP) downsamples the data after Conv1 for dimensionality reduction. The flattened output feeds into a fully connected layer, assigning weights to extracted features and predicting the image's class via softmax activation.

Optimization and Evaluation

Optimizer: Adam optimizer is utilized for parameter optimization. It adapts learning rates for each parameter based on gradient moments, enhancing convergence and performance.

Model Training: During training, the model iterates over mini-batches of the training dataset. Predictions are computed, and the loss is calculated using the Cross Entropy Loss formula. Backpropagation updates model parameters to minimize the loss.

Evaluation Metrics: Evaluation metrics, including train and test accuracy, precision, recall, and F1 score, assess model performance on both training and testing datasets. Accuracy measures the model's ability to correctly classify images by calculating the ratio of correctly predicted instances to the total instances. Precision indicates the accuracy of the positive predictions, recall measures the ability of the model to identify all relevant instances, and the F1 score provides a harmonic mean of precision and recall, offering a balanced measure of model performance.

Cross Entropy:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_i y_i \lg(\hat{y}_i)$$

where $\mathbf{y}$ is the true label distribution and $\hat{\mathbf{y}}$ is the predicted label distribution.

Accuracy Calculation:

$$Accuracy = \frac{Number of Correct Predictions}{Total Number of Predictions}$$

F1 score:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Recall (sensitivity):

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives}$$

Precision:

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives}$$

Model Saving: The best-performing model, determined by test accuracy, is saved for future deployment or fine-tuning, ensuring its usability in real-world applications. A small step by step description on how it works:

Input Layer: This layer resizes images to 200x200 pixels and normalizes them, preparing the data for further processing.

Convolutional Layers: The architecture includes three convolutional layers with increasing filter counts (12, 20, and 32) to capture various features. Each convolutional layer is followed by batch normalization and ReLU activation to enhance learning and model stability.

Pooling Layer: A Max Pooling layer is applied after the first convolutional layer to reduce the spatial dimensions, helping to manage computational complexity and focusing on the most prominent features.

Flattening: The 2D feature maps are converted into a 1D feature vector, making them suitable for the fully connected layer.

Fully Connected Layer: This layer maps the extracted features to the 43 output classes, completing the classification process.

Results:

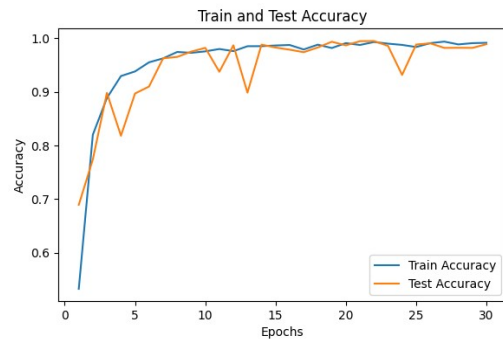


Fig 4: Train accuracy and Test accuracy

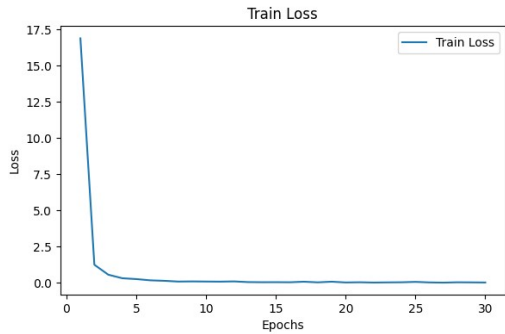


Fig 5: Loss vs Epoch number

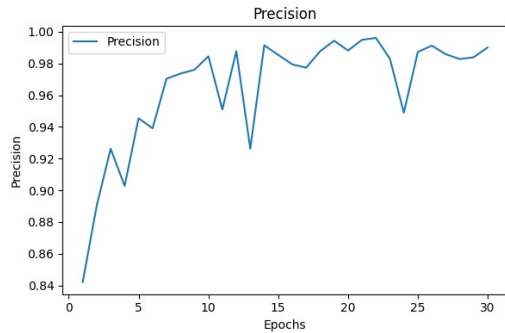


Fig 6: Precision vs Epoch number

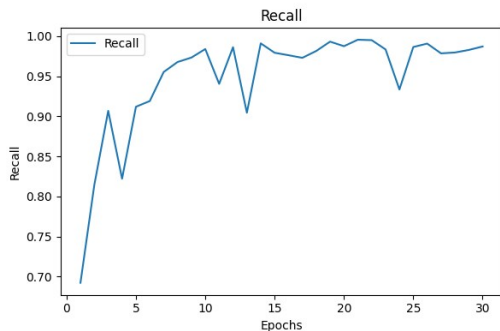


Fig 7: Recall vs Epoch number

Training and Testing Accuracy: The data presented in the graphs (fig. 4) depict the model's performance over 30 epochs. Initially, there is a rapid increase in accuracy. The training accuracy reaches 99.3%, while the testing accuracy peaks at 99.4%. The proximity of the lines indicating training and testing accuracy implies that the model generalizes well to new data without overfitting.

Training Loss: Looking at the loss graph (fig. 5), there is a significant decrease initially followed by consistently low values. The maximum training loss is 16.89%, with a minimum of 0.03%. This swift decline indicates efficient learning from the training data with minimal errors.

Precision: In terms of precision (fig. 6), the model starts at 85% and steadily climbs to over 95%, stabilizing at 98-99% with peaks at 99.6%. This high precision highlights the model's ability to accurately identify traffic signs with great precision.

Recall: Examining recall (fig. 7), we observe a gradual increase from approximately 70% to a consistent rate of 95-100%, reaching a high of 99.5%. This robust recall demonstrates the model's proficiency in correctly identifying all traffic signs, thereby reducing missed detections.

Analysis:

The Model's Performance and Observed Trends

The Convolutional Neural Network (CNN) designed for traffic sign recognition and classification performs exceptionally well. The rapid decrease in training loss and close alignment between training and testing accuracies underscore the model's strength and adaptability. These outcomes validate the model's capability to accurately detect and classify traffic signs, making it ideal for applications such as self-driving vehicles.

Potential Reasons for Misclassifications

The model operates efficiently most of the time, but occasionally makes errors, particularly when faced with signs that closely resemble each other. This issue is evident in both the visual representations and images where signs are inaccurately labeled. There are several factors that could contribute to misclassifications by the model:

Visual Similarity: Certain signs share similarities in shape, color, or design, making it challenging for the model to differentiate between them. This confusion results in errors in sign recognition.

Image Quality: In cases where the image is blurry, of poor quality, or inadequately lit, the model may struggle to identify the sign correctly.

Occlusions and Distortions: At times, objects such as trees or cars obstruct part of a sign, or the sign is captured from an unusual angle, leading to inaccuracies in identification by the model.

Insufficient Training Samples: Some categories of signs have limited examples for the model to learn from, causing it to overlook crucial details related to those signs.

Impact of Different Hyperparameters and Architectural Choices

The performance of the model is influenced by various hyperparameters and design choices. It is crucial to consider the following key factors:

Learning Rate & Optimizer: a learning rate set at 0.001 and utilizing the Adam optimizer are essential for effective learning and rapid convergence. Altering these parameters could impact the model's performance significantly.

Batch Size: The batch size of 256 for training and 128 for testing is recommended to achieve optimal results. Smaller batches may lead to noisy outcomes, while larger ones could slow down convergence.

Convolutional Layers & Kernel Size: The model consists of three convolutional layers with 5x5 kernel sizes aimed at capturing spatial patterns in images. Experimenting with different kernel sizes or adding more layers can enhance pattern extraction capabilities.

Pooling Layers: Max pooling layers aid in reducing the complexity of feature maps while retaining essential information. Exploring alternative pooling techniques such as average pooling may influence the model's performance.

Normalization & Augmentation: Implementing batch normalization and data augmentation techniques like random flips enhances the model's robustness and generalization abilities. Exploring additional methods for data augmentation or normalization can further elevate performance levels.

Data Set:

Size and Composition

The dataset utilized in this study addresses a single-image, multi-class classification problem, consisting of 43 distinct categories. It contains over 50,000 images in total, divided into approximately 39,000 images for training and 12,000 images for testing. The images are lifelike, reflecting various real-world conditions and scenarios, providing a robust basis for training and evaluating a Convolutional Neural Network (CNN).

Relevance to Problem

The dataset is crucial for training and assessing the CNN model's ability to recognize and classify images into one of the 43 predefined classes. The training dataset, comprising around 39,000 images across 43 classes, is used to train the CNN model, enabling it to learn the distinctive features and characteristics of each class. Initially, the testing dataset includes 12,000 unlabelled images, presenting a challenge for direct performance evaluation. To overcome this, a subset of images from each class in the training dataset was extracted to create a custom testing dataset with known classes, facilitating accurate performance measurement.

Ground Truth

The dataset includes CSV files providing the ground truth labels for both training and testing samples. These labels are essential for supervising the training process and validating the model's predictions during testing, ensuring the model's accuracy and reliability.



Fig. 5 Meta Datas of the classes of images that are used. There are 43 Types of traffic real life traffic signs that we used to train and evaluate the CNN model.

Conclusion:

The Convolutional Neural Network (CNN) design is an advanced and well-rounded approach for image categorization. By utilizing various convolutional layers, it effectively pulls out specific features from input images, like basic edges and intricate textures and shapes. Batch normalization adds stability and speeds up training, while

ReLU activations bring in necessary non-linearity to help the model learn complex patterns. Moreover, max pooling layers enhance efficiency by reducing spatial dimensions of feature maps. Standardized input resizing to 200x200 pixels guarantees consistency across the dataset, simplifying preprocessing and boosting model performance. The Adam optimizer with adaptive learning rates assists the model in quickly converging, optimizing performance and increasing reliability. This design isn't just strong but also adaptable for a variety of image categorization tasks beyond the current 43-class setup. Its high accuracy and metrics such as precision, recall, and F1 score showcase its ability to stand out in real-world applications like medical imaging and autonomous driving. All in all, this CNN design combines advanced technologies in feature extraction, normalization, and optimization to craft a potent tool for image categorization. Its well-rounded structure ensures top-notch performance, stability, and efficiency, making it a crucial tool for any task requiring precise and dependable image recognition capabilities.

Reference:

- [1] Fatin Zaklouta and Bogdan Stanculescu, 'Robotics and Autonomous Systems, Real-time traffic recognition in three stages', Robotics and Autonomous Systems 62(2014) 16-24.
- [2] P.G. Jimenez, S. Lafuente-Arroyo, H. Gomez-Moreno, F. Lopez-Ferreras, S. MaldonadoBascon, 'Traffic sign shape classification evaluation', part II, FFT applied to the signature of blobs, in Intelligent Vehicles Symposium, 2005, Proceedings, IEEE, IEEE, 2005, pp. 607– 612.
- [3] C. Fang, S. Chen, C. Fuh, 'Road-sign detection and tracking', IEEE Transactions on Vehicular Technology 52 (5) (2003) 1329–1341.
- [4] W.-J. Kuo, C.-C. Lin, 'Two-stage road sign detection and recognition', in 2007 IEEE International Conference on Multimedia and Expo, 2007, pp. 1427–1430.
- [5] Pierre Sermanet and Yann LeCun, 'Traffic Sign Recognition with Multi-Scale Convolutional Networks', Proceedings of International Joint Conference on Neural Networks, San Jose, California, USA, July 31 – August 5, 2011.
- [6] Yujun Zeng, Xin Xu, Yuqiang Fang, Kun Zhao, 'Traffic Sign Recognition Using Extreme Learning Classifier with Deep Convolutional Features'.
- [7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. 'Gradient-based learning applied to document recognition'. Proceedings of the IEEE, November 1998.
- [8] P. Arbelaez, J. Pont-Tuset, J. Barron, F. Marques, and J. Malik. Multiscale combinatorial grouping. In Computer Vision and Pattern Recognition (CVPR), 2014
- [9] J. Jin, K. Fu, and C. Zhang. Traffic sign recognition with hinge loss trained convolutional neural networks. IEEE Transactions on Intelligent Transportation Systems, 15(5):1991–2000, 2014.
- [10] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. CoRR, abs/1312.6229, 2013.